

MEMORIA DEL PROGRAMA GPS

Descripción de gps.py

El programa *gps.py* nos permite calcular rutas óptimas en el callejero de Madrid, ofreciendo al usuario tres tipos de rutas distintas: la más corta en distancia, la más rápida en tiempo y la más rápida considerando los tiempos esperados de semáforos. El programa está estructurado de manera que hace uso de módulos adicionales para organizar el código y sus distintas funcionalidades.

1. Módulos Implementados y Organización del Código

Al separar el código en módulos (*gps.py*, *callejero.py*, *grafo_pesado.py*), se mejora la claridad del programa. El programa consta de los siguientes módulos:

- i. ***gps.py***: el módulo principal. Interactúa con el usuario, le solicita las direcciones de origen y destino, y muestra las instrucciones de navegación y la ruta a seguir. Como se ha explicado antes, se ofrecen tres tipos distintos de rutas en función de lo que el usuario desee. En *gps.py*, se cuenta con diversas funciones necesarias para hacer los cálculos previos para encontrar toda la información necesaria de estas rutas.

- *calcula_peso_longitud(grafo, u, v)*
- *calcula_peso_tiempo(grafo, u, v)*
- *calcula_peso_tiempo_esperado(grafo, u, v)*
- *encuentra_nodo_mas_cercano(grafo, latitud, longitud)*

Además, se especifican las funciones en lo referente a la generación de instrucciones a seguir una ruta.

- *genera_instrucciones(grafo, ruta)*: imprime las indicaciones a seguir
- *resalta_ruta(grafo, ruta)*: dibuja la ruta elegida en el mapa

- ii. ***callejero.py***: Contiene funciones y clases auxiliares para la carga y procesamiento del callejero de Madrid, así como para la carga y procesamiento del grafo de la red vial. Este módulo cuenta con las siguientes funciones:

- *dms_to_decimal(dms_str)*: convierte coordenadas de formato DMS a decimales.
- *carga_callejero()*: **Cargar** el callejero desde un archivo CSV (*direcciones.csv*) descargado de la página web del ayuntamiento de Madrid, siendo este nuestro data frame.
- *busca_direccion(dirección, callejero)*: busca direcciones específicas y obtiene sus coordenadas de latitud y longitud, ya en forma decimal.

- *carga_grafo()*: carga el grafo de Madrid desde OpenStreetMap.
 - *procesa_grafo(grafo)*: convierte el grafo en un grafo dirigido sin bucles.
 - *dibuja_grafo(grafo_procesado)*: dibuja el grafo en base a sus coordenadas geográficas.
- iii. ***grafo_pesado.py***: incluye las implementaciones de los algoritmos de grafos necesarios para calcular las rutas de caminos mínimos entre dos nodos dados.
- *dijkstra(grafo, origen, peso)*: implementa **del** algoritmo de Dijkstra.
 - *camino_minimo(grafo, origen, destino, peso)*: calcula **al árbol** usando Dijkstra.
 - *prim(grafo, peso)*: implementa el algoritmo **Prim**
 - *kruskal(grafo, peso)*

2. Estructuras de Datos Utilizadas

El programa utiliza varias estructuras de datos para almacenar y manejar la información necesaria:

- **DataFrames de Pandas**: cargan y procesan el callejero de Madrid desde el archivo direcciones.csv.
- **Grafo de NetworkX**: utiliza un grafo dirigido (nx.DiGraph) para representar la red vial de Madrid. Los nodos representan intersecciones o puntos en las vías, y las aristas representan las conexiones entre ellos, con atributos como longitud, nombre de la calle o velocidad máxima.
- **GeoDataFrames de GeoPandas**: manejan las coordenadas geográficas y **proyectarlas** al sistema de coordenadas adecuado (CRS) para realizar cálculos espaciales precisos.
- **Diccionarios y Listas**: almacenan las distancias y padres en el algoritmo de Dijkstra, para construir y almacenar la ruta calculada y sus instrucciones.

3. Resolución de la Recuperación de la Ruta Óptima y Direcciones de Navegación

3.1 Funciones de Coste para los Tipos de Ruta

Para calcular los tres tipos de ruta solicitados, se han definido diferentes funciones de coste que se pasan al algoritmo de Dijkstra, ya mencionadas anteriormente.

- Ruta Más Corta (Distancia): utilizada cuando el usuario inserta la opción de calcular la ruta más corta en términos de distancia en la terminal, calcula el peso de una arista basado en su longitud en metros.

- Ruta Más Rápida (Tiempo): utilizada cuando el usuario inserta la opción de calcular la ruta más corta en términos de tiempo (sin tener semáforos en cuenta) en la terminal, calcula el peso de una arista basado en el tiempo estimado de viaje, considerando la longitud de la arista y su velocidad máxima permitida.
- Ruta Más Rápida con Semáforos (Tiempo Esperado): utilizada cuando el usuario inserta la opción de calcular la ruta más corta en términos de tiempo esperado contando semáforos, calcula el peso de una arista considerando el tiempo estimado de viaje, agregando un tiempo extra basado en la probabilidad de detenerse en un semáforo. **selecciona** la opción de ruta más rápida considerando el tiempo esperado en semáforos, proporcionando una estimación más realista del tiempo total de viaje.

3.2 Recuperación de la Ruta Óptima

El programa utiliza el algoritmo de Dijkstra para calcular la ruta óptima entre el origen y el destino, según la función de coste seleccionada. El proceso es el siguiente:

- i. Obtención de coordenadas de origen y destino: se solicitan al usuario las direcciones de origen y destino en formato "Tipo Calle Nombre Calle, Número". Haciendo uso de la función `busca_direccion` del módulo `callejero.py`, se obtienen las coordenadas geográficas (latitud y longitud) de ambas direcciones.
- ii. Proyección de coordenadas y encontrar nodos más cercanos: Las coordenadas geográficas se proyectan en el mismo plano que el grafo utilizando GeoPandas. **Utilizando** la función `ox.nearest_nodes` de OSMnx para encontrar los nodos del grafo más cercanos a las coordenadas proyectadas de origen y destino.
- iii. Cálculo de la ruta con Dijkstra: usando la función `camino_minimo` del módulo `grafo_pesado.py`, se da como argumentos el grafo, la función de coste calculada y los nodos de origen y destino. Así, el algoritmo calcula el camino mínimo basado en los pesos definidos por la función de costes.
- iv. Generación de instrucciones de navegación: se realiza mediante la función `genera_instrucciones`, que procesa la ruta y genera las indicaciones a seguir. Almacena la calle actual, la distancia acumulada y una lista para las instrucciones. Luego, se itera sobre los segmentos de la ruta y se obtienen los datos de la arista, con su nombre y longitud.
- v. Detección de cambios: agrega instrucciones nuevas cada vez que el nombre de la arista/calle cambia y acumula la distancia si el nombre de esta sigue siendo la misma.
- vi. Generación de instrucciones: crea instrucciones como "Continúe por la Calle ____ durante ____ metros" y agrega una instrucción final al llegar al destino.

- vii. Manejo de datos: existen casos en los que los atributos name o length pueden ser None. Así, se le asignan valores predeterminados como “vía desconocida”.

Conclusiones

El programa *gps.py* tiene el objetivo de calcular las rutas óptimas en el callejero de Madrid, en función de las preferencias del usuario. Proporciona instrucciones claras para el seguimiento del programa por parte del usuario, atendiendo a detalles que aseguran la precisión de las rutas.

Al separarlo en módulos y hacer uso de librerías, se consigue un código más eficiente, siendo fundamental el manejo de datos geográficos y la proyección de coordenadas.

Con este trabajo, conseguimos un gran manejo de los algoritmos de datos y grafos, obteniendo un programa útil para el cálculo de rutas de una gran ciudad como Madrid.

Bibliografía

Cao, María Belén (2024). Conocimientos aplicados en el desarrollo del trabajo sobre álgebra modular, Universidad Pontificia **de** Comillas, Escuela Superior de Ingeniería, ICAI.

Garicano, Carolina. (2024). Conocimientos aplicados en el desarrollo del trabajo sobre álgebra modular, Universidad Pontificia **de** Comillas, Escuela Superior de Ingeniería, ICAI.

Materiales del Tema 5 - Teoría de Grafos (2024). Transparencias de clase sobre teoría de grafos, **Matemáticas** Discreta, Universidad Pontificia de Comillas, Escuela Superior de Ingeniería, ICAI.

Materiales del Tema 6 – Algoritmos de recorrido de grafos (2024). Transparencias de clase sobre algoritmos de recorridos de grafos, Matemáticas Discreta, Universidad Pontificia de Comillas, Escuela Superior de Ingeniería, ICAI.

Documentación de NetworkX: <https://networkx.org/documentation/stable/>

OSMnx: Python for Street Networks: Boeing, G. (2017). *OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks*. Computers, Environment and Urban Systems, 65, 126-139. <https://osmnx.readthedocs.io/en/stable/>

Documentación de GeoPandas: <https://geopandas.org/>

Documentación de Pandas: <https://pandas.pydata.org/docs/>

ChatGPT (2024). Asistencia virtual en la elaboración de contenido. OpenAI, versión septiembre 2024.

Librería estándar de Python: <https://docs.python.org/3/library/>

Algoritmo de Dijkstra: Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1, 269-271.

Documentación de OpenStreetMap: Fuente de datos geográficos utilizada para el callejero de Madrid. <https://www.openstreetmap.org/>

Documentación de Matplotlib: Utilizada para la visualización gráfica de las rutas. <https://matplotlib.org/>